Class 19 Companion: The wine-wandb Project

Dataset, structure, files, and libraries

Data-driven Systems Engineering

What this deck covers

1. The dataset	Origin, the 11 measurements, the target, and the preparation decisions.
2. Project structure	Directories, root files, file types, and the role of each module.
3. How a run works	Configuration, the data flow, and the reproducibility safeguards.
4. Libraries	What each dependency does, with the calls the project actually uses.
5. Running it	Tests, continuous integration, and the commands.

The project in one sentence

A small, complete machine-learning project: a tracked baseline, a hyperparameter sweep, a versioned pipeline artifact, and a registry promotion, all for predicting red wine quality.

1. The dataset: Vinho Verde red wine

- **Source:** UCI Machine Learning Repository, *Wine Quality*; Cortez, Cerdeira, Almeida, Matos, Reis (2009), *Decision Support Systems*, 47(4), 547–553.
- **Content:** 1,599 red wines of the Portuguese Vinho Verde region, one row per wine.
- **Inputs:** 11 physicochemical measurements obtained in the laboratory.
- **Target:** `quality`, a sensory score from 0 to 10 given by wine experts (observed values: 3 to 8).
- **Format:** one CSV file, **semicolon-separated**; 12 columns, no missing values.

Two caveats worth discussing

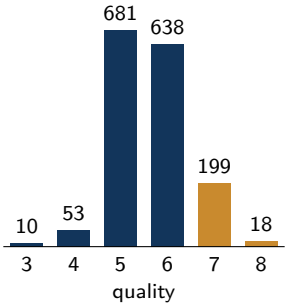
Duplicates: 240 rows are exact duplicates of another row. Copies that fall in both the training and the test split make the test score optimistic. **Subjectivity:** the target is an expert opinion, not a measurement; part of the error is irreducible.

The 11 input features

Column	Unit	Meaning	Median
`fixed acidity`	g/dm^3	Non-volatile acids (mainly tartaric)	7.9
`volatile acidity`	g/dm^3	Acetic acid; high values give a vinegar taste	0.52
`citric acid`	g/dm^3	Adds freshness and flavor	0.26
`residual sugar`	g/dm^3	Sugar left after fermentation	2.2
`chlorides`	g/dm^3	Salt content	0.079
`free sulfur dioxide`	mg/dm^3	Free SO_2 (preservative)	14
`total sulfur dioxide`	mg/dm^3	Free plus bound SO_2	38
`density`	g/cm^3	Close to water; depends on alcohol and sugar	0.997
`pH`	—	Acidity scale	3.31
`sulphates`	g/dm^3	Potassium sulphate; an additive	0.62
`alcohol`	% vol.	Alcohol content	10.2

Units and meanings follow the dataset documentation; medians were computed from the CSV.

The target and the two tasks



Number of wines per quality score.

Task 1: regression

Predict `quality` itself. Metric: R^2 (and MAE, RMSE). Used by the tracked baseline and the sweep.

Task 2: premium classification

Predict `quality` ≥ 7: 217 wines (13.6%). Metrics: precision, recall, F1, AUC. Used by the evaluation slides.

- Strongest linear association with quality (Pearson): alcohol $+0.48$, volatile acidity -0.39, sulphates $+0.25$, citric acid $+0.23$.
- Quality 3 and 8 are rare (10 and 18 wines); this is one reason the regression split is not stratified.

One dataset, two tasks, tracked the same way

	Task 1: regression	Task 2: premium classification
Question	What is the quality score?	Is the wine premium (quality ≥ 7)?
Target	`quality`, 3 to 8	217 of 1,599 wines (13.6%)
Model	`GradientBoostingRegressor`	`RandomForestClassifier`, 300 trees
Cross-validation	`KFold`, 5 folds	`StratifiedKFold`, 5 folds
Selection metric	Cross-validated R^2	Cross-validated F1-score
Decision rule	None: the output is a number	The predicted class (`predict`)
Tracked run	`train.py`, `sweep.py`	`train_classifier.py`
Artifact	`wine-pipeline`: `pipeline.joblib`	`wine-premium`: `pipeline.joblib`
Used in class for	Tracking, sweeps, registry	Evaluation, decision-support example

Why two tasks?

The same data answers two different questions. The choice of target space (a number or a class) changes the model and the metrics, and whether the output is already the answer or a class to act on.

Data preparation decisions and their reasons

Decision	Reason
Read with `sep=";"`	The UCI file uses semicolons, not commas.
Fixed 75/25 split, seed 42	1,199 training and 400 test wines; the same partition for every run, so scores are comparable.
Test split read only by `evaluate.py` and `figures.py`	Model selection uses cross-validation on the training split; the test split is evidence, not a tuning tool.
`StandardScaler` inside the pipeline	Fitted on the training folds only, so no preprocessing leakage. Gradient boosting does not need scaling; it is kept to show the pattern and to allow scale-sensitive models.
SHA-256 of the CSV logged per run	Identifies the exact dataset version used by each run.
CSV not committed to Git	Downloaded on demand from the UCI repository; avoids redistributing the file and keeps the repository small.

2. Project structure

```
wine-wandb/
|-- configs/                    baseline, sweep, classifier (YAML)
|-- src/wine_wandb/             the installable package
|   |-- data.py                 download, checksum, load, split
|   |-- pipeline.py             task 1: scaler + gradient boosting
|   |-- premium.py              task 2: label, classifier, metrics
|   |-- train.py                task 1: tracked run + artifact
|   |-- train_classifier.py     task 2: tracked run + artifact
|   |-- sweep.py                W&B grid search (task 1)
|   |-- evaluate.py             test split, once (both tasks)
|   |-- register.py             link to the Registry
|   `-- figures.py              figures for the slides
|-- tests/                      pytest
|-- data/                       CSV downloaded here (not versioned)
|-- notebooks/                  exploration only
|-- .github/workflows/          continuous integration
`-- README.md  requirements.txt  pyproject.toml  Makefile  .gitignore
```

Why each directory exists

Directory	Contains	Why it is separate
`configs/`	Seed, split, CV folds, default and swept hyperparameters (YAML)	Experiments change without touching code; a diff of a config is a diff of an experiment.
`src/wine_wandb/`	Importable Python modules	A package can be tested and reused; each module has one responsibility. The `src` layout prevents importing uninstalled code by accident.
`tests/`	Unit and smoke tests	Fast checks that run in CI without data or an account.
`data/`	The downloaded CSV	Raw data stays outside the code, and outside Git.
`notebooks/`	Exploratory notebooks	Exploration is allowed, but the reproducible logic lives in `src/`.
`.github/workflows/`	CI definition	Every push runs the tests.

Generated locally and ignored by Git: `wandb/` (run logs), `artifacts/` (downloaded artifacts), `reports/` (figures and JSON).

Root files

File	Type	Purpose
`README.md`	Markdown	Overview, the two tasks, quick start, and the checkpoints.
`requirements.txt`	Plain text	Runtime and test dependencies, each with a minimum version.
`pyproject.toml`	TOML	Package metadata (name, version, Python ≥ 3.10), the `src` layout, and the pytest settings.
`Makefile`	Make	Short commands: `install`, `data`, `train`, `train-classifier`, `sweep`, `evaluate`, `evaluate-premium`, `figures`, `test`.
`.gitignore`	Plain text	Keeps the CSV, `wandb/`, `artifacts/`, `reports/`, and environments out of Git.
`ci.yml`	YAML	GitHub Actions: install, then run pytest with `WANDB_MODE=disabled`.

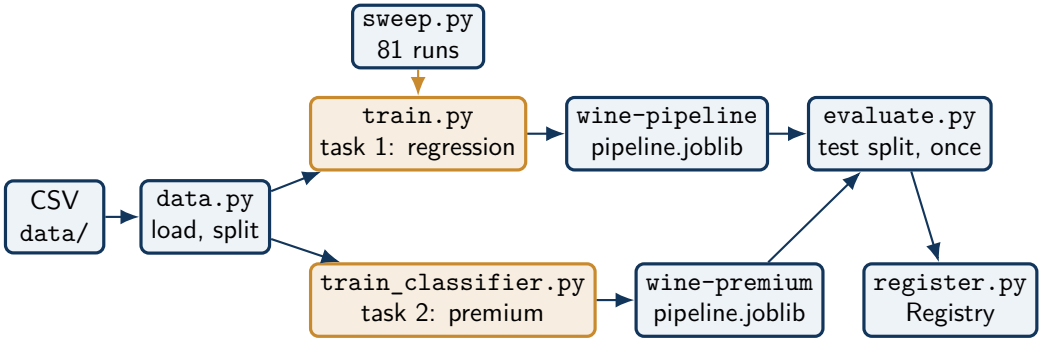
File types in the project

Extension	Format	Role
`.py`	Python source	Package modules and tests.
`.yaml / .yml`	YAML	Experiment configuration and the CI workflow; human-readable and diff-friendly.
`.csv`	Delimited text	The dataset (semicolon-separated); downloaded, never committed.
`.joblib`	Binary (pickle-based)	The serialized `Pipeline`; stored as a W&B artifact. Load only files you trust.
`.json`	JSON	`reports/results.json`: the measured numbers behind the slides.
`.pdf`	Vector graphics	Figures generated by `figures.py`.
`.md / .toml / .txt`	Text	Documentation, packaging metadata, and dependency list.
`.gitkeep`	Empty file	Keeps an otherwise empty directory (`data/`, `notebooks/`) in Git.

The package: one responsibility per module

Module	Responsibility	Main output
`data.py`	`download`, `sha256`, `load`, `split`	DataFrame; train and test partitions
`pipeline.py`	Task 1: scaler + `GradientBoostingRegressor`; rejects unknown hyperparameters	An unfitted `Pipeline`
`premium.py`	Task 2: label (quality ≥ 7), classifier pipeline, classification metrics	Pipeline; metrics
`train.py`	Task 1 run: CV score, timing, final fit	`wine-pipeline` artifact
`train_classifier.py`	Task 2 run: cross-validated F1, precision, recall, ROC-AUC; final fit	`wine-premium` artifact
`sweep.py`	Creates a sweep from `sweep.yaml`; the agent calls `train.run`	81 runs
`evaluate.py`	Scores an artifact on the test split (`-task regression` or `premium`)	`test/*` metrics
`register.py`	Links an artifact version to a Registry collection	Registry entry
`figures.py`	Test metrics of the classifier; overview and confusion-matrix figures	`reports/` (JSON, PDF)

Data flow: two tasks, one pattern



- Both runs log to W&B and produce a versioned pipeline artifact; the premium classifier uses its predicted class as the decision.
- Selection happens before `evaluate.py`: the test split never influences which artifact is chosen.

3. Configuration: baseline and sweep

configs/baseline.yaml

```yaml
project: wine-quality
seed: 42
test_size: 0.25
cv_folds: 5
n_estimators: 100
max_depth: 3
learning_rate: 0.1
min_samples_split: 2
```

configs/sweep.yaml

```yaml
method: grid
metric:
  name: cv/r2_mean
  goal: maximize
parameters:
  n_estimators:
    values: [100, 200, 300]
  max_depth:
    values: [3, 5, 7]
  learning_rate:
    values: [0.05, 0.1, 0.2]
  min_samples_split:
    values: [2, 5, 10]
```

- The baseline gives the defaults; during a sweep, W&B overrides the four hyperparameters and everything else (seed, split, folds) stays fixed.
- The sweep metric is the cross-validated R^2 on the training split, never the test split. Task 2 has its own `classifier.yaml` (seed, split, folds, number of trees).

Anatomy of one run (`train.run`)

1. Read the YAML defaults and download the CSV if it is missing.
2. `wandb.init(config=defaults)`: open the run; a sweep agent overrides the swept values.
3. Log the dataset SHA-256 in the run configuration.
4. Load the data and split it with the configured seed; keep only the training partition.
5. Build the pipeline from the four hyperparameters.
6. 5-fold `KFold` (shuffled, seeded) with `cross_val_score`, scoring R^2; log `cv/r2_mean`, `cv/r2_std`, and `cv/seconds`.
7. Refit the pipeline on the whole training split, save it with `joblib`, and log it as the `wine-pipeline` artifact together with its hyperparameters and CV score as metadata.

Why the refit?

Cross-validation estimates quality but leaves no model behind. The refit produces the single object that can be evaluated, registered, and served.

Anatomy of the classifier run (`train_classifier.run`)

1. Same start as task 1: YAML defaults, `wandb.init` with `job_type="train-classifier"`, and the dataset SHA-256 logged in the configuration.
2. Split with the same seed; turn `quality` into the premium label (`quality` ≥ 7).
3. `cross_validate` with `StratifiedKFold` (5 folds) and four scores: F1, precision, recall, and ROC-AUC; log their means, the F1 standard deviation, and the time.
4. Refit on the whole training split and log one artifact, `wine-premium`, with `pipeline.joblib`.

Why stratified folds and F1?

Premium wines are 13.6% of the data: stratification keeps that share in every fold, and F1 (unlike accuracy) is not inflated by the majority class.

Reproducibility safeguards

Risk	Safeguard in the project
Different data	SHA-256 of the CSV logged in every run configuration.
Different code	W&B records the Git commit of the repository where the script runs.
Different randomness	One seed for the split, the CV folds, and the model.
Preprocessing mismatch	Scaler and model travel together in one serialized `Pipeline`.
Preprocessing leakage	The scaler is refitted inside each training fold.
Test-set leakage	Only `evaluate.py` and `figures.py` read the test split; selection uses CV.
Silent regressions	Unit and smoke tests in CI; unknown hyperparameters raise an error.

4. Libraries and their roles

Library	Minimum	Role in the project
scikit-learn	1.4	Pipeline, preprocessing, models, cross-validation, and metrics.
pandas	2.1	Reads the CSV into a DataFrame; separates features and target.
NumPy	1.26	Numerical arrays: means and standard deviations of the CV scores.
joblib	1.3	Serializes the fitted pipeline to `pipeline.joblib`.
PyYAML	6.0	Reads the configuration files.
Matplotlib	3.8	Figures for the slides (non-interactive `Agg` backend).
wandb	0.17	Experiment tracking, sweeps, artifacts, and the Registry.
pytest	8.0	Test runner.

Standard library: `argparse` (command lines), `hashlib` (SHA-256), `urllib.request` (download), `pathlib`, `json`, `tempfile`, and `time`.

scikit-learn: the parts the project uses

Component	Use
`Pipeline`	Chains `StandardScaler` and the model into one estimator with `fit` and `predict`.
`StandardScaler`	Centers and scales each feature; parameters learned on the training data only.
`GradientBoostingRegressor`	The regression model; tuned over `n_estimators`, `max_depth`, `learning_rate`, `min_samples_split`.
`RandomForestClassifier`	The premium-wine classifier (300 trees) in `premium.py`.
`train_test_split`	The fixed, seeded 75/25 partition.
`KFold, StratifiedKFold`	Folds for regression and for the imbalanced premium label.
`cross_val_score`	Cross-validated R^2 for each configuration.
`cross_validate`	Several CV scores at once (F1, precision, recall, ROC-AUC) for the classifier.
`metrics`	`r2_score`, `mean_absolute_error`, `mean_squared_error`, `confusion_matrix`, `f1_score`, `roc_auc_score`, `average_precision_score`.

Weights & Biases: the calls the project makes

Call	Purpose	Module
`wandb.init`	Opens a run with its configuration; `job_type` labels non-training runs.	`train, evaluate, register`
`run.config.update`	Adds the dataset SHA-256 to the config.	`train`
`run.log`	Sends metrics (`cv/*`, `test/*`).	`train, evaluate`
`wandb.Artifact`	Creates a versioned `model` object with metadata.	`train, train_classifier`
`artifact.add_file`	Attaches `pipeline.joblib`.	`train, train_classifier`
`run.log_artifact`	Uploads it as a new version (`v0`, `v1`, ...).	`train, train_classifier`
`run.use_artifact`	Declares an input artifact (lineage); `.download()` fetches it.	`evaluate, register`
`run.link_artifact`	Publishes a version to a Registry collection.	`register`
`wandb.sweep, wandb.agent`	Define the grid and execute the runs.	`sweep`

Supporting libraries in context

pandas and NumPy

- `pd.read_csv(path, sep=";")` loads the data.
- `df.drop(columns="quality")` gives the features; `df["quality"]` the target.
- `np.mean`, `np.std` summarize the CV scores; `np.ndenumerate` labels the confusion matrix.

joblib and PyYAML

- `joblib.dump` / `joblib.load` store and restore the pipeline.
- `yaml.safe_load` reads configs without executing arbitrary tags.

Matplotlib

- `matplotlib.use("Agg")` renders to files, so figures can be produced on a server or in CI.
- Two PDF figures: dataset overview and confusion matrix.

pytest

- Plain `assert` statements; fixtures build synthetic data.
- `monkeypatch` sets `WANDB_MODE=disabled` in the smoke test.

5. Tests and continuous integration

File	Tests	What it checks
`test_pipeline.py`	4	Split is reproducible and sized correctly; the pipeline fits and predicts; it works inside cross-validation; unknown hyperparameters are rejected.
`test_train_smoke.py`	1	`train.run` completes end to end on synthetic data, with W&B disabled.
`test_premium.py`	3	The premium label; the counts and metrics of a small example (including ROC-AUC); the classifier pipeline fits and predicts.
`test_train_classifier_smoke.py`	1	`train_classifier.run` completes end to end on synthetic data, with W&B disabled.

- The tests need neither the dataset download nor a W&B account, so they run anywhere.
- The GitHub Actions workflow installs the requirements and runs `pytest` on every push and pull request (Ubuntu, Python 3.11, `WANDB_MODE=disabled`).
- The sweep, evaluation, and registration need a W&B account and are not part of CI. In total, 9 tests.

Running the project

```
python -m venv .venv && source .venv/bin/activate
make install
wandb login                          # once; not needed for the tests

make data                            # download the CSV
make train                           # task 1: baseline run
make sweep                           # task 1: 81 configurations
make train-classifier                # task 2: premium classifier
make evaluate ARTIFACT=wine-pipeline:v0          # test split, once
make evaluate-premium ARTIFACT=wine-premium:v0   # test split, once
make figures                         # dataset overview and confusion matrix
make test                            # no account required
```

- Choose the artifact version by cross-validation; run the matching `evaluate` target once; then link it with `python -m wine_wandb.register -artifact <name:version> -collection <registry path>`.

Summary

What the project demonstrates

- A **dataset** with known limits (subjective target, duplicates) and explicit preparation decisions.
- A **structure** where configuration, code, tests, data, and generated files each have a place.
- Two **tracked runs** (regression and premium classification) that record data, code, configuration, metrics, and a versioned pipeline.
- **Libraries** chosen for one job each, with the minimum versions declared.

Connection to the next class

The next class presents Streamlit and the dataset; the registered pipeline is the artifact an interface would load.